

The "65536 transformations" in this algorithm is a combinatorial explosion of function compositions, rather than 65,536 unique mathematical formulas. The core mathematical novelty is the massive multi-layered search space, which uses 256 deterministic, lossless base transforms, a 2-byte XOR preprocessor, an LZ77+Huffman+RLE pipeline, and a min-select backend compression, all wrapped in a brute-force verification framework.

Here is the full mathematical presentation of the UnifiedPAQJP+PJP algorithm. By Jurijus Pacalovas

1. The Base Transform Functions ($\mathcal{T}$)

Let $\mathbb{B}^*$ represent the set of all finite byte sequences $\{0, 1\}^8$.

The algorithm defines a finite set of 256 base transform functions:

$$\mathcal{T} = \{ T_1, T_2, T_3, \dots, T_{256} \}$$

Where every $T_i: \mathbb{B}^* \rightarrow \mathbb{B}^*$ is a deterministic, bijective, and invertible function (i.e., $\forall D, T_i^{-1}(T_i(D)) = D$).

These functions fall into five main mathematical categories:

1. Bitwise & Arithmetic (XOR, Rotations, Modular Add/Subtract): e.g., $T_{21}(D) = (D + 255) \bmod 256$ (per byte), $T_{46}(D) = D \oplus M$ (XOR with a fixed periodic mask M).

2. Finite Field Exponentiation ($\mathbb{F}_{257}$): e.g., T_{35} applies $y = (x+1)^3 \bmod 257$ -

1. This exploits the prime modulus property for bijectivity.

3. Substitution/Permutation: e.g., $T_{\{06\}}$ is a fixed 256-byte pseudo-random permutation S-box.
4. Compressive Tokenization: e.g., $T_{\{27\}}$ maps the data into a 6-bit alphabet dictionary (pure bijection), $T_{\{45\}}$ is a canonical Huffman encoder with an exact header.
5. Identity: $T_{\{256\}}(D) = D$. This acts as a null transform.

2. The 65,535 Transform Pairs (Ω_{pairs})

The algorithm's 65,535 variations are ordered pairs of the base transforms.

The composition space is defined as:

$$\mathcal{P} = \{ (T_a, T_b) \mid a, b \in \{1, 2, \dots, 256\} \}$$

The total number of possible unique pairs is $256^2 = 65,536$. The code intentionally excludes the single trivial pair $(T_{\{256\}}, T_{\{256\}})$, resulting in exactly:

$$|\mathcal{P}_{\text{exclusive}}| = 256^2 - 1 = 65,535$$

For any input data D , applying a specific pair $p = (T_a, T_b)$ is mathematically defined as the function composition:

$$F_p(D) = T_b(T_a(D))$$

Deep Ultra Mode (Multi-Pair Sequences):

The algorithm further extends this to k-length sequences ($k \geq 2$):

$$F_{\{(p_1, p_2, \dots, p_k)\}}(D) = \left(T_{\{b_k\}} \circ T_{\{a_k\}} \circ \dots \circ T_{\{b_1\}} \circ T_{\{a_1\}} \right)(D)$$

This yields an immense search space:

- $|\mathcal{P}|^2 \approx 4.3 \times 10^9$ variations (Two pairs)
- $|\mathcal{P}|^3 \approx 2.8 \times 10^{14}$ variations (Three pairs)

3. The 2-Byte Preprocessor ($\mathbf{Pre}_{2b}$)

Before the transform search, the data is optionally passed through a 2-byte XOR preprocessor to increase compressibility.

Let $L = \text{len}(D)$ and $\text{parity} = L \bmod 2$.

If $\text{parity} = 1$, pad D with 0x00 to ensure L is even.

We define the set of all 16-bit little-endian words in D :

$$W = \{ \text{int}_{LE}(D[i:i+2]) \mid 0 \leq i < L, i \equiv 0 \pmod{2} \}$$

The optimal XOR key K is calculated using the mode (most frequent value):

$$K = \underset{k \in W}{\operatorname{argmax}} \left(\sum_{w \in W} \mathbf{1}_{\{k = w\}} \right)$$

The preprocessed data is then:

$$D_{\{\text{pre}\}} = \text{bigoplus}_{w \in W} (w \oplus K)$$

(Here bigoplus means concatenating the transformed 16-bit blocks to form the new byte sequence).

4. The LZH / RLE / Backend Pipeline

When the "LZH" or "Ultra LZH" modes are selected, the selected $F_p(D)$ output is passed through a final lossless compression pipeline:

A. LZ77 Sliding Window Parser:

The data is parsed into a sequence of tokens $t = \{ \text{Literal}, (\text{Distance}, \text{Length}) \}$.

For a match at position i , the parser maximizes:

$$\max_{j \in [\max(0, i-w), i)} \{ L \mid D[j:j+L] = D[i:i+L], L \geq 3 \}$$

Where $W=2048$ is the window size and $L_{\text{max}}=2048$.

B. Static Canonical Huffman Coding:

The algorithm counts the frequencies of literals (F_{lit}), distances (F_{dist}), and lengths (F_{len}). It generates the minimum-redundancy prefix code lengths via a Huffman algorithm:

$$\text{Lengths}(X) = \text{Huffman}(F_X)$$

It then maps these into canonical binary codes C_x where the code length is $l_x = \text{Lengths}(X)[x]$.

C. Byte-Level Run-Length Encoding (RLE):

The Huffman bitstream is encoded into bytes, and a byte-level RLE is applied:

$$\begin{aligned} \text{RLE}(B) = & \begin{cases} \\ \text{[0xFF, run-1, byte]} \ \& \ \text{if run} \geq 4 \ \text{or byte} = 0xFF \ \backslash \\ \text{[byte]} \ \$\times\$ \text{run} \ \& \ \text{otherwise} \\ \end{cases} \\ & \end{cases}$$

D. Backend Compression:

The algorithm acts as a selector between two optional backends and raw output.

Let $\text{end}(X)$ represent the compressed output of backend X .

The compressed candidate is:

$$\text{Result} = \min_{\text{len}} \left(\text{Zstd}(X), \text{PAQ}(X), \text{Raw}(X) \right)$$

To ensure exact decoding, this is prepended with a one-byte header $\text{marker} \in \{0x00, 0x01, 0x02\}$.

5. The Global Compression Objective Function

The entire algorithm is a lossless combinatorial search with strict verification.

Let Θ be the total search space defined as:

$$\Theta = \{ \text{Raw} \} \cup \{ T_i \} \cup \{ T_a \circ T_b \} \cup \{ T_{b_k} \circ T_{a_k} \dots \}$$

The optimal compressed output C^* is determined by finding the shortest length while satisfying the exact equality constraint:

$$C^*(D) = \underset{C \in \Theta}{\operatorname{argmin}} \left(\text{len}(C(D)) \right) \quad \text{s.t.} \quad \text{Decompress}(C(D)) = D$$

Because Θ cannot be traversed entirely for the 281 trillion variations (Deep Ultra), the algorithm uses a time-limited heuristic random search ($\text{RNG based on seed } 42$) to explore the combinatorial explosion within time_limit seconds.

Decompression Verification:

Every successful compression candidate is immediately checked by running the inverse sequence:

$$D_{\{\text{restored}\}} = T_{\{\text{first}\}}^{-1} \left(T_{\{\text{second}\}}^{-1} \left(\dots \right. \right. \\ \left. \left. \text{Backend}^{-1}(C^*) \right) \right)$$

If $D_{\{\text{restored}\}} \neq D$, the candidate is discarded, guaranteeing 100% data integrity across all modes.

Summary Presentation

- Core Math: 256 Finite-Invertible Transforms.
- Combinatorics: 65,535 Ordered Pairs (built via $T_i \circ T_j$).
- Expansion: Deep Ultra allows up to 3 pairs, reaching $\sim 2.8 \times 10^{14}$ possibilities.
- Preprocessing: Mode-based 16-bit XOR masking to flatten byte frequencies.
- Pipeline: LZ77 search $\rightarrow$ Huffman canonical prefixes $\rightarrow$ Byte-RLE $\rightarrow$ Min-select Zstd/PAQ fallback.

Because the fundamental building blocks are pure mathematical inverses (like modular rotations, finite-field exponentiations, and strict prefix-free Huffman codes), the entire 65,536-tier search space is lossless by structural design, further bolstered by a final post-compression byte-for-byte verification pass.